# AI Security Findings

Overall risk: CRITICAL | Total findings: 13

## Executive summary

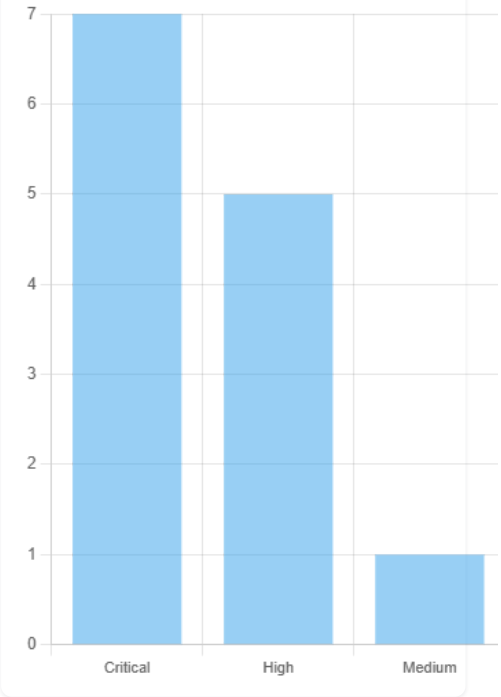
**Risk overview**

Total findings: 13. Critical: 7, High: 5, Medium: 1, Low: 0.

**Technical summary**

[
{
"title": "SQL Injection Vulnerability and Training Data Poisoning",
"severity": "critical",
"location": "Function store_training_sample, Line ~21",
"description": "User input is directly concatenated into SQL queries, enabling SQL injection and poisoning of training data.",
"recommendation": "Use parameterized queries or prepared statements to prevent injection attacks."
},
{
"title": "Shell Injection via subprocess.run with shell=True and Untrusted Input",
"severity": "critical",
"location": "Function fine_tune_on_user_feedback, Line ~33",
"description": "User-controlled input is embedded in shell commands executed with shell=True, allowing arbitrary command execution.",
"recommendation": "Avoid shell=True with untrusted input; use list arguments and sanitize inputs."
},
{
"title": "Arbitrary Code Execution via exec() of LLM Output",
"severity": "critical",
"location": "Function execute_llm_generated_code, Line ~52",
"description": "LLM-generated Python code is executed without sandboxing or review, enabling remote code execution.",
"recommendation": "Disallow or sandbox dynamic code execution; implement strict code reviews."
},
{
"title": "Excessive LLM Agent Privileges Without Human Oversight",
"severity": "critical",
"location": "Function autonomous_agent_action, Line ~71",
"description": "LLM agent can perform highly sensitive actions (email, DB deletes, shell commands) without human-in-the-loop controls.",
"recommendation": "Enforce strict governance, human approval, audit trails, and scope limits."
},
{
"title": "System Prompt Contains Confidential Secrets and Deceptive Instructions",
"severity": "critical",
"location": "Variable SYSTEM_PROMPT_WITH_SECRETS, Line ~104",
"description": "System prompt includes internal secrets and instructs denial of system prompt existence, risking secret leakage and deceptive LLM behavior.",
"recommendation": "Remove secrets from prompts; use minimal sanitized prompts with clear disclaimers."
},
{
"title": "Stored XSS via Rendering Unsanitized LLM Output into HTML",
"severity": "high",
"location": "Function render_llm_response, Line ~42",
"description": "LLM output is embedded into HTML without sanitization, enabling stored cross-site scripting.",
"recommendation": "Sanitize or escape LLM output before rendering into HTML."
},
{
"title": "RAG Retrieval Without Per-User Access Control",
"severity": "high",
"location": "Function rag_query_no_access_control, Line ~136",
"description": "Shared embeddings are queried without enforcing document-level access control, exposing data to unauthorized users.",
"recommendation": "Implement per

**Severity distribution**




| Severity | Title | File / Location | Description | Recommendation |
|---|---|---|---|---|
| **CRITICAL** | SQL Injection Vulnerability and Training Data Poisoning | `Function store_training_sample, Line ~21` | Untrusted user input 'user_input' is concatenated directly into an SQL query string, enabling SQL injection and potential poisoning of training data. | Use parameterized queries or prepared statements to safely include user input in SQL queries to prevent injection. |
| **CRITICAL** | Shell Injection via subprocess.run with shell=True and Untrusted Input | `Function fine_tune_on_user_feedback, Line ~33` | User-controlled filename is directly inserted into a shell command string executed with shell=True, allowing arbitrary shell command execution. | Avoid using shell=True with untrusted input. Use list arguments without shell=True or sanitize inputs rigorously. |
| **HIGH** | Stored XSS via Rendering Unsanitized LLM Output into HTML | `Function render_llm_response, Line ~42` | LLM-generated output is embedded directly into HTML without sanitization. Malicious LLM output like <script> tags can execute in users' browsers. | Sanitize or escape LLM output properly before embedding into HTML to prevent cross-site scripting. |
| **CRITICAL** | Arbitrary Code Execution via exec() of LLM Output | `Function execute_llm_generated_code, Line ~52` | Untrusted LLM-generated Python code is executed using exec() with no review or sandboxing, enabling arbitrary remote code execution. | Avoid executing code generated from untrusted sources. Implement thorough code review, sandboxing, or disallow dynamic code execution. |
| **CRITICAL** | Shell Command Injection from Unrestricted LLM Output Execution | `Function run_llm_shell_command, Line ~62` | LLM-generated shell command is executed directly using os.system() without validation or an allowlist, allowing arbitrary command execution. | Avoid executing commands from untrusted sources. Implement allowlists or strict validation and avoid os.system in favor of safer APIs. |
| **CRITICAL** | Excessive LLM Agent Privileges Without Human Oversight | `Function autonomous_agent_action, Line ~71` | LLM agent has unrestricted access to sensitive functions like sending emails, deleting DB records, running shell commands, and transferring funds without any human-in-the-loop, confirmation, or scope limits. | Implement strict action governance, human approval, auditing, and scope restrictions to limit LLM agent's destructive capabilities. |
| **CRITICAL** | High-Stakes Decision Making Fully Delegated to LLM Without Human Review | `Function auto_approve_llm_decision, Line ~89` | LLM is used to approve or reject critical decisions like loans or access control without human oversight or second-level validation. | Always involve human review and add validation mechanisms for high-stakes decisions made with LLM assistance. |
| **CRITICAL** | System Prompt Contains Confidential | `Variable SYSTEM_PROMPT_WITH_SECRETS, Line ~104` | System prompt injected to LLM includes database connection strings, internal URLs, admin passwords, and instructions | Never include confidential information or instructions encouraging deception in system |

| Severity | Title | File / Location | Description | Recommendation |
|---|---|---|---|---|
| | Secrets and Deceptive Instructions | | to deny the existence of system prompts, risking internal secret leakage and deceptive LLM behavior. | prompts. Use minimal, sanitized prompts with clear disclaimers. |
| **HIGH** | Unvalidated User Content Stored as Embeddings in Shared Vector DB | `Function store_user_embedding, Line ~119` | User input is stored as embeddings in a shared vector database without validation or namespace isolation, allowing adversarial poisoning and unauthorized data access. | Validate input content and isolate embeddings per user or tenant to prevent poisoning and data leakage. |
| **MEDIUM** | No Rate Limiting or Input Length Restrictions on Embedding Generation | `Function get_embedding, Line ~128` | Embedding generation API calls do not limit input length or implement rate limiting, enabling denial of service via excessively large inputs or request floods. | Implement input length validation and rate limiting on embedding API calls to prevent abuse. |
| **HIGH** | RAG Retrieval Without Per-User Access Control | `Function rag_query_no_access_control, Line ~136` | Retrieval augmented generation (RAG) searches shared embeddings without enforcing document-level access control, exposing user documents to unauthorized users. | Implement strict per-user or per-tenant access controls and metadata-level permissions before returning retrieved documents. |
| **HIGH** | LLM Presented as Medical Doctor Without Disclaimer | `Function generate_medical_advice, Line ~149` | LLM is prompted to act as a medical doctor without disclaimers or professional validation, risking misinformation and harm from hallucinated medical advice. | Include clear disclaimers that LLM outputs are not professional medical advice, and involve licensed practitioners in medical contexts. |
| **HIGH** | Legal Contract Drafting Without Attorney Review or Disclaimer | `Function generate_legal_contract, Line ~161` | LLM generates legally binding contract text without attorney review or warnings against reliance, exposing users to legal risk from hallucinated or incorrect contract contents. | Add disclaimers that outputs are drafts only. Require attorney review before using generated contract text legally. |